1. Fundamental Questions

1. What is System Design?
2. Why is System Design important?
3. What is scalability?
4. Difference between horizontal and vertical scaling.
5. What is load balancing?
6. What is high availability?
7. What is fault tolerance?
8. What is redundancy?
9. What is reliability?
10. What is latency?
11. What is throughput?
12. What is bandwidth?
13. What is bottleneck?
14. What is single point of failure (SPOF)?
15. What is a distributed system?

---

2. Architecture Questions

1. What is client-server architecture?
2. What is a three-tier architecture?
3. Difference between monolithic and microservices.
4. What are APIs?
5. What is reverse proxy?
6. Difference between forward proxy and reverse proxy.

---

3. Database Questions

1. SQL vs NoSQL.
2. What is database indexing?
3. What is normalization?
4. What is denormalization?

5. What is sharding?
6. What is replication?
7. Primary key vs Foreign key.
8. ACID properties.
9. CAP theorem.
10. Eventual consistency.

---

4. Caching Questions

1. What is caching?
2. Why is caching used?
3. Types of cache.
4. What is cache eviction?
5. Explain LRU cache.
6. Cache hit vs cache miss.
7. Write-through vs Write-back cache.

---

5. Networking Questions

1. What is DNS?
2. What is HTTP?
3. Difference between HTTP and HTTPS.
4. TCP vs UDP.
5. What is SSL/TLS?
6. What is CDN?
7. What is a firewall?
8. What is NAT?
9. What is a socket?
10. What is a port?

---

6. Load Balancer Questions

1. Why use a load balancer?
2. Types of load balancing.
3. Round Robin algorithm.
4. Least Connection algorithm.
5. Health checks.

6. Sticky sessions.

---

7. Storage Questions

1. File storage vs Block storage vs Object storage.
2. What is RAID?
3. What is SAN?
4. What is NAS?
5. What is SSD?
6. What is cloud storage?

---

8. Security Questions

1. Authentication vs Authorization.
2. OAuth.
3. JWT.
4. Session management.
5. Encryption vs Hashing.
6. Role-Based Access Control (RBAC).

---

9. Interview MCQ/Concept Questions

1. Why is a CDN used?
2. Why is Redis faster than a database?
3. When should NoSQL be preferred?
4. Why use a Load Balancer?
5. What happens if a database server fails?
6. Why do we shard databases?
7. Why use caching before a database?
8. What is eventual consistency?
9. What is idempotency?
10. What is rate limiting?

---

10. Practical Scenario Questions

1. How would you design a system to support **1 million users**?

2. How would you reduce response time from 5 seconds to 500 milliseconds?

3. What happens when traffic suddenly increases 100×?

4. How would you prevent database overload?

5. How would you make a system highly available?

6. How would you store billions of images?

7. How would you handle server failures?

8. How would you scale an e-commerce website during a festival sale?

9. How would you detect duplicate requests?